Assignment 1: Tomasulo Algorithm Implementation

Oyekunle Oshidele, ID-2143251

October 20, 2026

1 Introduction

Tomasulo algorithm. Tomasulo's algorithm schedules instructions dynamically: instructions issue in order, execute out of order as soon as their operands are available, and commit in order through the reorder buffer (ROB). Register renaming through the RAT removes WAR and WAW hazards, and the load/store queue performs memory disambiguation.

In this assignment I did the following:

1. I modified the existing Python Tomasulo simulator so that it **issues, writes back and commits up to *W* instructions per cycle, in order**. W is the issue width, set to 1–4 in the input file (`Issue width = N`, default 1). All test cases default to **issue width 3**, and every test case is also run with widths 1, 2 and 4 for comparison.
2. I made the hardware configuration parameterizable from the input file (number of RS, EX/MEM cycles, number of FUs, ROB entries, issue width, initial registers and memory). The simulator now prints the final register values and the non-zero memory values.
3. I wrote six test cases, a sequential reference interpreter and an automatic test runner. For every test case and issue width, the runner checks the final architectural state and the pipeline timing rules (Section 3).

What works

- Issue, write-back (CDB) and commit of 1–4 instructions per cycle, in program order.
- Register renaming of dependent instructions *inside the same issue group*. Instructions are renamed one after another within the cycle, so a younger instruction sees the RAT updates of the older ones.
- In-order issue stalls: issue stops at the first instruction that finds its RS, ld/sd queue or ROB full, and younger instructions wait.
- Branches (`Bne` and `Beq`) work for every issue width: fetch stops after a branch and restarts the cycle after the branch is resolved in EX.
- Pipelined FP adder and FP multiplier, unpipelined integer adder, and more than one FU of each type (`# of FUs`).
- Load/store queue with store-to-load forwarding. Stores write memory at commit through the single-ported, non-pipelined memory.

- Final register file and non-zero memory values are printed, and they match the sequential reference for all 24 runs (6 tests × 4 widths).

Limitations

- There is no branch prediction or speculative fetch past a branch. The original code did not have a predictor or BTB either; it stalled fetch until the branch resolved, and I kept that behaviour. Because of this, no wrong-path instructions are ever fetched, so no squash or RAT recovery is needed.
- Loads use conservative memory disambiguation. A load waits while an older store still has an unknown address, instead of speculatively accessing memory.

2 Code Changes

I started with the given Python codes of the algorithm and added more codes to enable multi-issue. The changes are described below.

2.1 Input file and issue width (`config.py` (new), `main.py`)

The original `main.py` hard-coded the configuration and read only the instructions from `code.in`. I added `config.py`, which reads the assignment's input format, and `main.py` now takes the test file and an optional width override:

```
python main.py ../tests/test3_branch_loop.txt          # width from the file
python main.py ../tests/test3_branch_loop.txt -w 3     # override (1-4)
```

```
match = re.match(r'issue width\s*=\s*(\d+)', lower)
if match:
    cfg['issue_width'] = int(match.group(1))
...
def check_config(cfg):
    if not 1 <= cfg['issue_width'] <= 4:
        raise ValueError('issue width must be between 1 and 4')
```

The parser is case insensitive and accepts commas between operands, as the assignment requires. For example, `add.d f4, f2, f1` is normalized to `Add.d F4 F2 F1`.

2.2 Multiple issue (`issue.py`)

Initial version: one instruction was fetched and issued per call.

```
def issue(cycle, PC, instructions, ROB, size_ROB, rs_int_adder, rs_fp_adder,
          rs_fp_multi, ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    ...
```

My version: the old body became `issue_one()`, which returns whether the instruction was issued. `issue()` calls it up to `issue_width` times in program order and stops at the first structural hazard, at a branch, or at the end of the program.

```
def issue(cycle, PC, instructions, ROB, size_ROB, rs_int_adder, rs_fp_adder,
          rs_fp_multi, ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp, issue_width):
    # fetch and issue up to issue_width instructions in program order
    for i in range(issue_width):
        # end of program, or fetch stalled by an unresolved branch
        if (PC.PC>=len(instructions))|(PC.valid==0)|(cycle<PC.resume_cycle):
            break
        # structural hazard: younger instructions must wait (in-order issue)
        if not issue_one(cycle, PC, instructions, ROB, size_ROB, ...):
            break
```

Each instruction updates the RAT before the next instruction in the same group reads it, so RAW dependences inside a group are renamed correctly. For example, `Add R3 R1 R2` → `Sub R4 R3 R1` → `Add R3 R4 R4` are all issued in cycle 1 in Test 5. A branch sets `PC.valid = 0`, which ends the group.

2.3 Multiple write-back (`wb.py`)

Initial version: the CDB carried one result per cycle, and a result fetched from the results buffer was broadcast only in the *next* cycle.

```
if cdb.valid == 1:
    boroadcast(...)
    cdb.valid = 0
if cdb.valid == 0:
    if len(results_buffer)>0:
        cdb.valid = 1
        ...
```

My version: up to `issue_width` results are broadcast per cycle, one CDB per issue slot.

```
bd_count = 0
while (len(results_buffer)>0)&(bd_count<issue_width):
    cdb.valid = 1
    cdb.value = results_buffer[0].value
    cdb.dest_tag = results_buffer[0].dest_tag
    results_buffer.popleft()
    boroadcast(cdb, rat_int, rat_fp, rs_int_adder, rs_fp_adder, rs_fp_multi,
               ld_sd_queue, ROB, cycle)
    cdb.valid = 0
    bd_count += 1
```

The broadcast also records `wake_cycle = cycle` in every RS or ld/sd queue entry it wakes up. The EX stage uses this to make sure an operand written back in cycle n is used in EX in cycle $n+1$ at the earliest.

2.4 Multiple commit (`commit.py`)

Initial version: at most one instruction committed per cycle, plus a special case for a store at the head. Branches were removed without recording a commit cycle.

My version: up to `issue_width` instructions commit per cycle, in order, and commit stops at the first head instruction that is not ready.

```
for i in range(issue_width):
    if len(ROB)==0:
        break
    head = ROB[0]
```

```
    op = instructions[head.PC].split(' ')[0]
    if (op=='Bne')|(op=='Beq'):
        # resolved in an earlier cycle
        if (len(head.exe)<2) or (head.exe[-1]>=cycle):
            break
    elif op=='Sd':
        entry = find_ldsd_entry(ld_sd_queue, head.ROB_tag)
        # address and data ready, memory port free
        if (entry.ready==0) or (... data not ready ...) or (ld_sd_mem.busy==1):
            break
        # dequeue the Sd and write memory
        ld_sd_queue.remove(entry)
        put_entry_into_ld_sd_mem(ld_sd_mem, entry)
        head.mem.extend([cycle, cycle+time_ld_sd_mem-1])
    else:
        # broadcasted in an earlier cycle
        if (len(head.cdb)==0) or (head.cdb[0]>=cycle):
            break
        modify_arch_reg(head, reg_int, reg_fp)
    head.commit.append(cycle)
    committed.append(ROB.popleft())
```

Stores now write memory **at commit**, as the assignment requires. Because memory is single-ported, at most one store commits per cycle.

2.5 Execution stage (`exe.py`)

Initial version: `fu_exe` advanced every instruction in a FU, and was then called a *second* time in the same cycle when a waiting instruction was moved from an RS into the FU. Instructions already in a pipelined FU therefore advanced two cycles in one (the sample report fixed this with a call-history dictionary). Only one instruction per FU type could start per cycle, and the lowest-numbered RS entry was chosen rather than the oldest.

My version: in every cycle, ready instructions are first moved from the RS into the FUs, and then every instruction in a FU advances **exactly once**:

```
def fetch_rs_into_fu(rs, fu, num_fu, pipelined, ROB, cycle):
    if pipelined:
        slots = num_fu              # each unit accepts one instruction per cycle
    else:
        slots = num_fu - len(fu)    # unpipelined: only idle units accept one
    for i in range(slots):
        index = check_valid_ins_in_rs(rs, ROB, cycle)
        ...

def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    for element in list(fu):
        element.cycle += 1
        ...
        if element.cycle == time_fu:
            ...
            fu.remove(element)
```

`check_valid_ins_in_rs` now picks the **oldest** ready instruction (smallest ROB position) whose operands were not broadcast in this cycle (`wake_cycle < cycle`). The integer adder is unpipelined and the FP adder and multiplier are pipelined, as the assignment specifies, and `# of FUs` > 1 is supported. Branch resolution moved to `resolve_branch()`, which supports `Beq` as well as `Bne`, computes the target from the branch's own PC, and restarts fetch in the next cycle (`PC.resume_cycle = cycle + 1`).

2.6 Memory stage (`mem.py`)

- **Store-to-load forwarding** now searches older stores from the nearest to the farthest. A load is forwarded from the nearest matching store once that store has its data. It waits if an older store address is unknown or the matching store has no data yet. Otherwise it goes to memory. The original code only looked at the nearest store, regardless of its address.
- Stores are no longer sent to memory by the MEM stage. In the original code the "will this store commit?" check was `if (check_if_sd_committable):`, which tests the *function object* and is always true, so stores wrote memory before they committed.
- A load sent to memory starts its access in the same cycle (`[start, start+Mem-1]`), and its result is written back the cycle after the access ends.

2.7 Order of the pipeline stages (`main.py`)

As in the sample report, the stages are now called in reverse pipeline order inside each cycle:

```
'''COMMIT stage'''  commit(...)
'''CDB stage'''     wb(...)
'''MEM stage'''     mem(...)
'''EXE stage'''     exe(...)
'''ISSUE stage'''   issue(...)
```

Each stage therefore only sees what the later pipeline stages produced in previous cycles, so for every instruction ISSUE $<$ EX $\leq$ EX end $<$ MEM $<$ WB $<$ COMMIT holds without extra bookkeeping. This is what allows W instructions to move through each stage in the same cycle. The loop now also runs until the last store has finished writing memory, not just until the ROB is empty.

3 Results

3.1 How correctness is checked

`tests/run_tests.py` runs every test case with issue widths 1, 2, 3 and 4 and checks each run in two ways:

1. **Functional check.** `tests/reference.py` executes the program sequentially, one instruction at a time. The simulator's final integer registers, FP registers and all 256 bytes of memory must equal the reference, and so must the *committed instruction stream* (the sequence of PCs, which checks that branches go the right way).
2. **Pipeline-rule check.** Issue and commit happen in program order. No cycle has more than W issues, W write-backs or W commits. For every instruction ISSUE $<$ EX start $\leq$ EX end, Ld: EX $<$ MEM $<$ WB $<$ COMMIT, ALU: EX $<$ WB $<$ COMMIT, branch: EX $<$ COMMIT, and Sd: EX $<$ COMMIT $=$ MEM start.

All 24 runs pass:

Test case	Instr.	W=1 cycles (IPC)	W=2 cycles (IPC)	W=3 cycles (IPC)	W=4 cycles (IPC)	Check
test1_sample_report_case1	17	36 (0.472)	33 (0.515)	33 (0.515)	33 (0.515)	PASS
test2_multiple_fus	17	47 (0.362)	46 (0.370)	46 (0.370)	46 (0.370)	PASS
test3_branch_loop	15	41 (0.366)	35 (0.429)	33 (0.455)	32 (0.469)	PASS
test4_store_load_forwarding	11	21 (0.524)	20 (0.550)	20 (0.550)	20 (0.550)	PASS
test5_dependencies_in_group	20	38 (0.526)	35 (0.571)	33 (0.606)	33 (0.606)	PASS
test6_structural_hazards	11	30 (0.367)	31 (0.355)	31 (0.355)	31 (0.355)	PASS

(Cycles = the cycle of the last commit; IPC = committed instructions / cycles.)

3.2 Test Case #1 (Test Case #01 of the sample report)

Configuration

```
               # of rs          Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        4               3                               1
FP multiplier   4               5                               1
Load/store unit 4               1               6               1
ROB entries = 64
Issue width = 3
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Instructions

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Issue width 1. This is identical, cycle for cycle, to Figure 1 of the sample report, which validates the new stage order against the original single-issue behaviour:

```
                  ISSUE       EXE         MEM         WB          COMMIT
Ld F1 0(R1)       [1]         [2, 2]      [3, 8]      [9]         [10]
Mult.d F2 F1 F0   [2]         [10, 14]    []          [15]        [16]
Add.d F3 F3 F2    [3]         [16, 18]    []          [19]        [20]
Mult.d F2 F1 F1   [4]         [11, 15]    []          [17]        [21]
Add.d F4 F2 F4    [5]         [18, 20]    []          [21]        [22]
Addi R1 R1 8      [6]         [7, 7]      []          [8]         [23]
Ld F1 0(R1)       [7]         [9, 9]      [10, 15]    [16]        [24]
Mult.d F2 F1 F0   [8]         [17, 21]    []          [23]        [25]
Add.d F3 F3 F2    [9]         [24, 26]    []          [27]        [28]
Mult.d F2 F1 F1   [10]        [18, 22]    []          [24]        [29]
Add.d F4 F2 F4    [11]        [25, 27]    []          [28]        [30]
Addi R1 R1 8      [12]        [13, 13]    []          [14]        [31]
Ld F1 0(R1)       [13]        [15, 15]    [16, 21]    [22]        [32]
Mult.d F2 F1 F0   [14]        [23, 27]    []          [29]        [33]
Add.d F3 F3 F2    [16]        [30, 32]    []          [33]        [34]
Mult.d F2 F1 F1   [17]        [24, 28]    []          [30]        [35]
Add.d F4 F2 F4    [18]        [31, 33]    []          [34]        [36]
```

Instructions committed: 17, cycles: 36, IPC: 0.472

Issue width 3:

```
                    ISSUE       EXE         MEM         WB          COMMIT
Ld F1 0(R1)         [1]         [2, 2]      [3, 8]      [9]         [10]
Mult.d F2 F1 F0     [1]         [10, 14]    []          [15]        [16]
Add.d F3 F3 F2      [1]         [16, 18]    []          [19]        [20]
Mult.d F2 F1 F1     [2]         [11, 15]    []          [16]        [20]
Add.d F4 F2 F4      [2]         [17, 19]    []          [20]        [21]
Addi R1 R1 8        [2]         [3, 3]      []          [4]         [21]
Ld F1 0(R1)         [3]         [5, 5]      [9, 14]     [15]        [21]
Mult.d F2 F1 F0     [3]         [16, 20]    []          [21]        [22]
Add.d F3 F3 F2      [3]         [22, 24]    []          [25]        [26]
Mult.d F2 F1 F1     [4]         [17, 21]    []          [22]        [26]
Add.d F4 F2 F4      [4]         [23, 25]    []          [26]        [27]
Addi R1 R1 8        [4]         [5, 5]      []          [6]         [27]
Ld F1 0(R1)         [5]         [7, 7]      [15, 20]    [21]        [27]
Mult.d F2 F1 F0     [10]        [22, 26]    []          [27]        [28]
Add.d F3 F3 F2      [16]        [28, 30]    []          [31]        [32]
Mult.d F2 F1 F1     [16]        [23, 27]    []          [28]        [32]
Add.d F4 F2 F4      [17]        [29, 31]    []          [32]        [33]
```

Instructions committed: 17, cycles: 33, IPC: 0.515

This table is identical, cycle for cycle, to the issue-width-3 table (Figure 3) of the sample report. Three instructions issue per cycle in cycles 1–4. In cycle 5 only the third `Ld` issues: the next instruction is the fifth `Mult.d`, and all 4 FP-multiplier RS are occupied. It issues in cycle 10, when the first `Mult.d` starts executing and frees its RS. Commit is limited by the long Ld → Mult.d → Add.d dependence chain, so widths 2–4 all finish in 33 cycles, compared with 36 for width 1.

3.3 Test Case #2 (multiple FUs, Test Case #02 of the sample report)

Configuration

```
               # of rs          Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        3               3                               2
FP multiplier   2               15                              2
Load/store unit 4               1               5               1
ROB entries = 128
Issue width = 3
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Instructions

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Addi R1 R1 4
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
```

```
Add.d F4 F2 F4
```

Issue width 3:

```
                    ISSUE       EXE         MEM         WB          COMMIT
Ld F1 0(R1)         [1]         [2, 2]      [3, 7]      [8]         [9]
Mult.d F2 F1 F0     [1]         [9, 23]     []          [24]        [25]
Add.d F3 F3 F2      [1]         [25, 27]    []          [28]        [29]
Mult.d F2 F1 F1     [2]         [9, 23]     []          [24]        [29]
Ld F1 0(R1)         [2]         [3, 3]      [8, 12]     [13]        [29]
Addi R1 R1 4        [2]         [3, 3]      []          [4]         [30]
Add.d F4 F2 F4      [3]         [25, 27]    []          [28]        [30]
Addi R1 R1 8        [3]         [5, 5]      []          [6]         [30]
Ld F1 0(R1)         [3]         [7, 7]      [13, 17]    [18]        [31]
Mult.d F2 F1 F0     [9]         [19, 33]    []          [34]        [35]
Add.d F3 F3 F2      [9]         [35, 37]    []          [38]        [39]
Mult.d F2 F1 F1     [9]         [19, 33]    []          [34]        [39]
Ld F1 0(R1)         [10]        [11, 11]    [18, 22]    [23]        [39]
Mult.d F2 F1 F0     [19]        [24, 38]    []          [39]        [40]
Add.d F3 F3 F2      [25]        [40, 42]    []          [43]        [44]
Mult.d F2 F1 F1     [25]        [26, 40]    []          [41]        [44]
Add.d F4 F2 F4      [25]        [42, 44]    []          [45]        [46]
```

Instructions committed: 17, cycles: 46, IPC: 0.370

With two FP multipliers, both `Mult.d` that read `F1` start in the same cycle (9, then 19) and write back in the same cycle (24, 34). The 3-wide CDB carries both results, and up to three instructions commit in one cycle (cycles 29, 30 and 39). The 15-cycle multiplies dominate, so the gain over width 1 is only one cycle.

3.4 Test Case #3 (branch loop, the provided `test_case.txt`)

Configuration

```
                # of rs          Cycles in EX     Cycles in Mem    # of FUs
Integer adder    4               1                                 1
FP adder         3               4                                 1
FP multiplier    2               15                                1
Load/store unit 5                1                5                1
ROB entries = 64
Issue width = 3
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Instructions

```
Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -12
Add.d F20 F2 F2
```

The `Bne` jumps back 3 instructions (to `Addi R1`) while `R1 != 0`, so the loop body runs 3 times: 15 dynamic instructions.

Issue width 1:

```
                    ISSUE       EXE         MEM         WB          COMMIT
```

```
Ld F2 0(R1)         [1]         [2, 2]      [3, 7]      [8]         [9]
Mult.d F4 F2 F20    [2]         [9, 23]     []          [24]        [25]
Ld F6 0(R2)         [3]         [4, 4]      [8, 12]     [13]        [26]
Add.d F6 F4 F6      [4]         [25, 28]    []          [29]        [30]
Sd F6 0(R2)         [5]         [6, 6]      [31, 35]    []          [31]
Addi R1 R1 -4       [6]         [7, 7]      []          [9]         [32]
Addi R2 R2 -4       [7]         [8, 8]      []          [10]        [33]
Bne R1 R0 -12       [8]         [10, 10]    []          []          [34]
Addi R1 R1 -4       [11]        [12, 12]    []          [14]        [35]
Addi R2 R2 -4       [12]        [13, 13]    []          [15]        [36]
Bne R1 R0 -12       [13]        [15, 15]    []          []          [37]
Addi R1 R1 -4       [16]        [17, 17]    []          [18]        [38]
Addi R2 R2 -4       [17]        [18, 18]    []          [19]        [39]
Bne R1 R0 -12       [18]        [19, 19]    []          []          [40]
Add.d F20 F2 F2     [20]        [21, 24]    []          [25]        [41]
```

Instructions committed: 15, cycles: 41, IPC: 0.366

Issue width 3:

```
                    ISSUE       EXE         MEM         WB          COMMIT
Ld F2 0(R1)         [1]         [2, 2]      [3, 7]      [8]         [9]
Mult.d F4 F2 F20    [1]         [9, 23]     []          [24]        [25]
Ld F6 0(R2)         [1]         [3, 3]      [8, 12]     [13]        [25]
Add.d F6 F4 F6      [2]         [25, 28]    []          [29]        [30]
Sd F6 0(R2)         [2]         [4, 4]      [30, 34]    []          [30]
Addi R1 R1 -4       [2]         [3, 3]      []          [4]         [30]
Addi R2 R2 -4       [3]         [4, 4]      []          [5]         [31]
Bne R1 R0 -12       [3]         [5, 5]      []          []          [31]
Addi R1 R1 -4       [6]         [7, 7]      []          [8]         [31]
Addi R2 R2 -4       [6]         [8, 8]      []          [9]         [32]
Bne R1 R0 -12       [6]         [9, 9]      []          []          [32]
Addi R1 R1 -4       [10]        [11, 11]    []          [12]        [32]
Addi R2 R2 -4       [10]        [12, 12]    []          [13]        [33]
Bne R1 R0 -12       [10]        [13, 13]    []          []          [33]
Add.d F20 F2 F2     [14]        [15, 18]    []          [19]        [33]
```

Instructions committed: 15, cycles: 33, IPC: 0.455

At width 3, each loop iteration (`Addi, Addi, Bne`) issues in one cycle (6 and 10). Fetch then waits until the `Bne` is resolved (EX 9 → next issue 10, EX 13 → next issue 14). The whole program takes 33 cycles instead of 41. Final state (width 3; it is the same for every width):

```
REGISTER VALUES
        R0          R1          R2          R3          R4          R5          R6          R7
value:  0           0           20          0           0           0           0           0
        R8          R9          R10         R11         R12         R13         R14         R15
value:  0           0           0           0           0           0           0           0
        R16         R17         R18         R19         R20         R21         R22         R23
value:  0           0           0           0           0           0           0           0
        R24         R25         R26         R27         R28         R29         R30         R31
value:  0           0           0           0           0           0           0           0
        F0          F1          F2          F3          F4          F5          F6          F7
value:  0           0           1.0         0           3.0         0           7.0         0
        F8          F9          F10         F11         F12         F13         F14         F15
value:  0           0           0           0           0           0           0           0
        F16         F17         F18         F19         F20         F21         F22         F23
value:  0           0           0           0           2.0         0           0           0
        F24         F25         F26         F27         F28         F29         F30         F31
value:  0           0           0           0           0           0           0           0

NON-ZERO MEMORY VALUES
Addresses   Values
4           3.0
8           2.0
12          1.0
```

```
24          6.0
28          5.0
32          7.0
```

`F2 = Mem[12] = 1.0`, `F4 = 1.0 x 3.0`, `F6 = 3.0 + Mem[32] = 7.0` (stored to `Mem[32]`), `R1 = 12 - 3*4 = 0`, `R2 = 32 - 3*4 = 20`, `F20 = 1.0 + 1.0`. All of these are correct.

3.5 My test cases

Test Case #4: store-to-load forwarding and memory disambiguation

Configuration

```
               # of rs          Cycles in EX    Cycles in Mem   # of FUs
Integer adder   3               1                               1
FP adder        3               3                               1
FP multiplier   2               6                               1
Load/store unit 6               1               4               1
ROB entries = 32
Issue width = 3
R1=8, R2=16, F1=1.5, F2=2.5
Mem[8]=10.0, Mem[16]=20.0, Mem[24]=30.0
```

Instructions

```
Mult.d F3 F1 F2
Sd F3 0(R1)
Ld F4 0(R1)
Ld F5 8(R1)
Addi R3 R2 8
Sd F4 0(R3)
Ld F6 24(R0)
Add.d F7 F4 F6
Sd F7 4(R2)
Ld F8 20(R0)
Add.d F9 F8 F5
```

This test checks that loads in the same issue group as older stores get the right value:

- `Ld F4 0(R1)` reads the address that `Sd F3 0(R1)` stores to, while the store's data (from a 6-cycle multiply) is not ready yet. The load must wait, then **forward** (1-cycle MEM).
- `Ld F5 8(R1)` has a different address and goes to memory.
- `Sd F4 0(R3)` stores a value that is itself a forwarded load, through a base register (`R3`) produced by an `Addi` in the same group.
- `Ld F6 24(R0)` and `Ld F8 20(R0)` use R0 bases that match the stores `0(R3)` (=24) and `4(R2)` (=20), so they must forward from them.

Issue width 3:

```
                  ISSUE     EXE       MEM       WB        COMMIT
Mult.d F3 F1 F2   [1]       [2, 7]    []        [8]       [9]
Sd F3 0(R1)       [1]       [2, 2]    [9, 12]   []        [9]
Ld F4 0(R1)       [1]       [3, 3]    [8]       [9]       [10]
Ld F5 8(R1)       [2]       [4, 4]    [5, 8]    [9]       [10]
Addi R3 R2 8      [2]       [3, 3]    []        [4]       [10]
Sd F4 0(R3)       [2]       [5, 5]    [13, 16]  []        [13]
Ld F6 24(R0)      [3]       [6, 6]    [9]       [10]      [13]
```

```
Add.d F7 F4 F6      [3]         [11, 13]    []          [14]        [15]
Sd F7 4(R2)         [3]         [7, 7]      [17, 20]    []          [17]
Ld F8 20(R0)        [5]         [8, 8]      [14]        [15]        [17]
Add.d F9 F8 F5      [5]         [16, 18]    []          [19]        [20]
```

Instructions committed: 11, cycles: 20, IPC: 0.550

The three forwarded loads show a single MEM cycle (`[8]`, `[9]`, `[14]`), and `Ld F5` uses the memory port (`[5, 8]`). Each store writes memory at its commit cycle, one store at a time (`[9,12]`, `[13,16]`, `[17,20]`).

```
REGISTER VALUES
        R0        R1        R2        R3        R4        R5        R6        R7
value:  0         8         16        24        0         0         0         0
        R8        R9        R10       R11       R12       R13       R14       R15
value:  0         0         0         0         0         0         0         0
        R16       R17       R18       R19       R20       R21       R22       R23
value:  0         0         0         0         0         0         0         0
        R24       R25       R26       R27       R28       R29       R30       R31
value:  0         0         0         0         0         0         0         0
        F0        F1        F2        F3        F4        F5        F6        F7
value:  0         1.5       2.5       3.75      3.75      20.0      3.75      7.5
        F8        F9        F10       F11       F12       F13       F14       F15
value:  7.5       27.5      0         0         0         0         0         0
        F16       F17       F18       F19       F20       F21       F22       F23
value:  0         0         0         0         0         0         0         0
        F24       F25       F26       F27       F28       F29       F30       F31
value:  0         0         0         0         0         0         0         0

NON-ZERO MEMORY VALUES
Addresses   Values
8           3.75
16          20.0
20          7.5
24          3.75
```

Test Case #5: dependences inside one issue group, R0, Beq/Bne

Configuration

```
               # of rs          Cycles in EX     Cycles in Mem    # of FUs
Integer adder   4                2                                1
FP adder        3                2                                1
FP multiplier   2                4                                1
Load/store unit 4                1                3               1
ROB entries = 64
Issue width = 3
R1=3, R2=5, F1=2.0, F2=0.5
```

Instructions

```
add r3, r1, r2
sub r4, r3, r1
add r3, r4, r4
addi r0, r3, 7
mult.d f3, f1, f2
add.d f3, f3, f1
sub.d f4, f3, f2
addi r5, r0, 8
addi r5, r5, -4
add.d f4, f4, f1
sd f4, 0(r5)
bne r5, r0, -16
```

```
beq r5, r0, 4
addi r6, r6, 100
addi r7, r7, 1
ld f6, 0(r0)
add.d f7, f6, f4
```

This test uses lower-case instructions with commas (to check the parser) and contains:

- a RAW chain inside one issue group (`add r3` → `sub r4` → `add r3`, all issued in cycle 1), together with WAW on `r3`;
- a write to `r0`, which must be discarded;
- a WAW/RAW chain on `f3`/`f4`;
- a loop through `bne` (taken once) and a taken `beq` that skips `addi r6`;
- a load from the address that the last store wrote.

The integer adder is unpipelined with 2 EX cycles, so the effect of an unpipelined unit is visible.

Issue width 3:

	ISSUE	EXE	MEM	WB	COMMIT
Add R3 R1 R2	[1]	[2, 3]	[]	[4]	[5]
Sub R4 R3 R1	[1]	[6, 7]	[]	[8]	[9]
Add R3 R4 R4	[1]	[10, 11]	[]	[12]	[13]
Addi R0 R3 7	[2]	[14, 15]	[]	[16]	[17]
Mult.d F3 F1 F2	[2]	[3, 6]	[]	[7]	[17]
Add.d F3 F3 F1	[2]	[8, 9]	[]	[10]	[17]
Sub.d F4 F3 F2	[3]	[11, 12]	[]	[13]	[18]
Addi R5 R0 8	[3]	[4, 5]	[]	[6]	[18]
Addi R5 R5 -4	[4]	[8, 9]	[]	[10]	[18]
Add.d F4 F4 F1	[4]	[14, 15]	[]	[16]	[19]
Sd F4 0(R5)	[4]	[11, 11]	[19, 21]	[]	[19]
Bne R5 R0 -16	[6]	[12, 13]	[]	[]	[19]
Addi R5 R5 -4	[14]	[16, 17]	[]	[18]	[20]
Add.d F4 F4 F1	[14]	[17, 18]	[]	[19]	[20]
Sd F4 0(R5)	[14]	[19, 19]	[22, 24]	[]	[22]
Bne R5 R0 -16	[15]	[19, 20]	[]	[]	[22]
Beq R5 R0 4	[21]	[22, 23]	[]	[]	[24]
Addi R7 R7 1	[24]	[25, 26]	[]	[27]	[28]
Ld F6 0(R0)	[24]	[25, 25]	[26, 28]	[29]	[30]
Add.d F7 F6 F4	[24]	[30, 31]	[]	[32]	[33]

Instructions committed: 20, cycles: 33, IPC: 0.606

`Sub R4` waits for `Add R3` (WB 4), and because the integer adder is unpipelined and busy with `Addi R5` in cycles 4–5, it starts in cycle 6. `Addi R6` is never fetched: the `Beq` is taken. The final `R0` is 0 and `R6` is 0:

```
REGISTER VALUES
        R0      R1      R2      R3      R4      R5      R6      R7
value:  0       3       5       10      5       0       0       1
        R8      R9      R10     R11     R12     R13     R14     R15
value:  0       0       0       0       0       0       0       0
        R16     R17     R18     R19     R20     R21     R22     R23
value:  0       0       0       0       0       0       0       0
        R24     R25     R26     R27     R28     R29     R30     R31
value:  0       0       0       0       0       0       0       0
        F0      F1      F2      F3      F4      F5      F6      F7
value:  0       2.0     0.5     3.0     6.5     0       6.5     13.0
        F8      F9      F10     F11     F12     F13     F14     F15
value:  0       0       0       0       0       0       0       0
```

```
        F16       F17       F18       F19       F20       F21       F22       F23
value:  0         0         0         0         0         0         0         0
        F24       F25       F26       F27       F28       F29       F30       F31
value:  0         0         0         0         0         0         0         0

NON-ZERO MEMORY VALUES
Addresses   Values
0           6.5
4           4.5
```

Test Case #6: structural hazards

Configuration

```
               # of rs          Cycles in EX    Cycles in Mem   # of FUs
Integer adder   1                1                               1
FP adder        1                3                               1
FP multiplier   1                5                               1
Load/store unit 2                1               2               1
ROB entries = 4
Issue width = 3
R1=4, F1=1.0, F2=2.0
Mem[4]=5.0, Mem[8]=6.0
```

Instructions

```
Ld F3 0(R1)
Ld F4 4(R1)
Add.d F5 F3 F4
Mult.d F6 F5 F2
Add.d F7 F6 F1
Addi R2 R1 8
Sd F7 0(R2)
Mult.d F8 F1 F2
Addi R3 R2 4
Sd F8 0(R3)
Ld F9 0(R3)
```

Every unit has a single reservation station, the ld/sd queue has 2 entries, and the ROB has only 4 entries. Issue must stall in order whenever any of them is full, at every issue width.

Issue width 3:

```
                   ISSUE      EXE         MEM         WB          COMMIT
Ld F3 0(R1)        [1]        [2, 2]      [3, 4]      [5]         [6]
Ld F4 4(R1)        [1]        [3, 3]      [5, 6]      [7]         [8]
Add.d F5 F3 F4     [1]        [8, 10]     []          [11]        [12]
Mult.d F6 F5 F2    [2]        [12, 16]    []          [17]        [18]
Add.d F7 F6 F1     [8]        [18, 20]    []          [21]        [22]
Addi R2 R1 8       [8]        [9, 9]      []          [10]        [22]
Sd F7 0(R2)        [12]       [13, 13]    [22, 23]    []          [22]
Mult.d F8 F1 F2    [18]       [19, 23]    []          [24]        [25]
Addi R3 R2 4       [22]       [23, 23]    []          [24]        [25]
Sd F8 0(R3)        [22]       [25, 25]    [26, 27]    []          [26]
Ld F9 0(R3)        [22]       [26, 26]    [28, 29]    [30]        [31]
```

Instructions committed: 11, cycles: 31, IPC: 0.355

Here a wider issue does **not** help: width 1 takes 30 cycles and widths 2–4 take 31. With width ≥ 2, `Sd F8 0(R3)` commits earlier (cycle 26) and leaves the load/store queue before `Ld F9 0(R3)` reaches its MEM stage. The load can therefore no longer be forwarded from the store and has to wait for the memory port (`[28, 29]`). With width 1, the store is still in the queue and the load

is forwarded (MEM `[28]`). The results are still correct (same final state), and this is a real timing effect of the "stores leave the queue at commit" policy.

3.6 Discussion

- The final state is identical for widths 1–4 in all tests, as is the committed instruction stream.
- The speed-up from a wider issue is largest in code with short, independent integer instructions and branches (Test 3: 41 → 33 cycles, −20%; Test 5: 38 → 33). It is smallest when long FP dependence chains (Tests 1 and 2) or a single memory port and tiny RS/ROB (Test 6) are the bottleneck.
- Going from width 3 to 4 helps only in Test 3 (33 → 32), because these short programs rarely have four ready instructions in the same cycle.

4 How to run

```
cd code
python main.py                                  # test_case.txt, issue width 3
python main.py ../tests/test5_dependencies_in_group.txt -w 2
cd ../tests
python run_tests.py                             # all tests, widths 1-4, writes tests/
    results/
```

Requires Python 3 and no external packages.